Création et utilisation de la base de données

Anthony Rake

Laplace Immo

Contexte du projet

- Objectif : exploiter les données immobilières
- Projet DATAImmo piloté par la CTO
- mieux **comprendre le marché immobilier**
- aider à **estimer les prix des biens**

La stratégie de sauvegarde et la conformité RGPD

Pour garantir la sécurité et la disponibilité des données :

- Données issues de sources open data publiques (DVF, INSEE…)
- La table DVF contient le nom de l’acquéreur. Cette colonne a été écartée lors de la création des tables finales dans le respect des principes du RGPD : minimisation des données, finalité (analyse immobilière), absence de données sensibles.
- Il conviendra de placer les répertoires de données dans un espace serveur sécurisé afin d’en limiter l’accès (RGPD).

Les données initiales

- DVF (Demandes de Valeurs Foncières) : transactions immobilières (prix, date, type de bien…)
- INSEE : données de population par commune
- Référentiel géographique (data.gouv) : communes, départements, régions

Caractéristiques et enjeux

Caractéristiques :

- Données brutes et hétérogènes
- Formats différents selon les sources
- Présence de redondances et variables inutiles

Enjeux :

- Nettoyer et structurer les données
- Harmoniser les formats
- Préparer l’intégration dans une base relationnelle

L’extrait du dictionnaire des données

Table Bien :

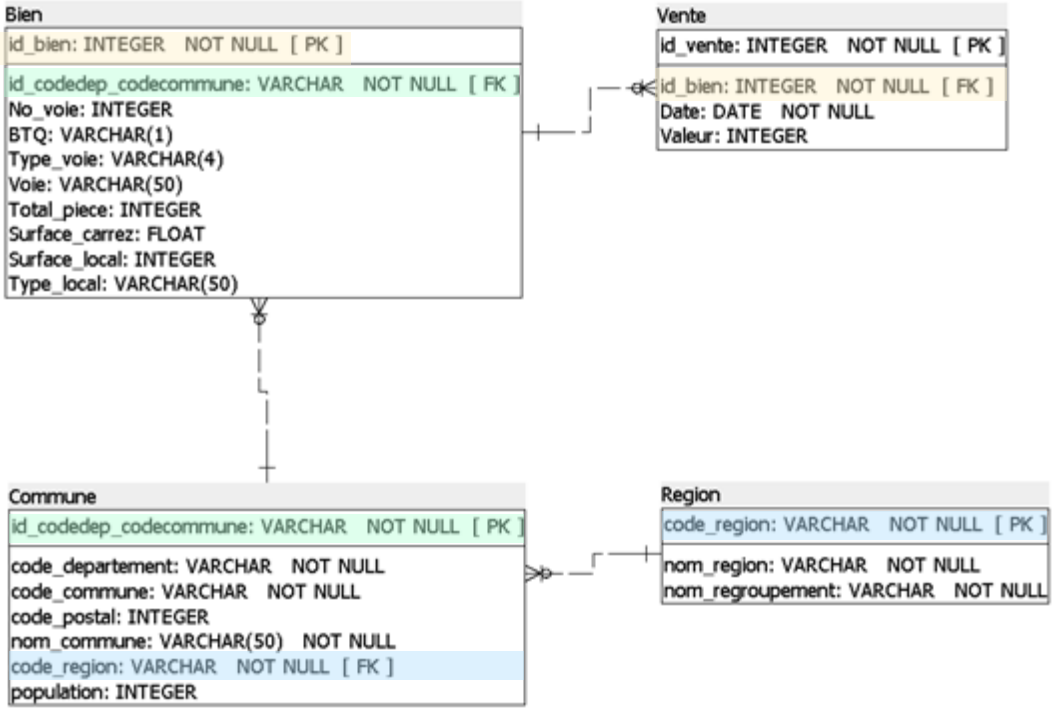
CODE	SIGNIFICATION	TYPE	LONGUEUR	NATURE	REGLE DE GESTION	REGLE DE CALCUL
id_bien	identifiant unique du bien (PK)	Integer	NC	Incrémentation	Not null	1 à N
id_codedep_codecommune	identifiant unique dépatement + commune (FK)	Varchar	5	Concaténation	Not null	code_departement + code_commune avec suffixe 0
No_voie	numéro de voie	Integer	NC	Elémentaire		
BTQ	indice de répétition bis ter quarter	Varchar	1	Elémentaire		
Type_voie	type de voie	Varchar	4	Elémentaire		
Voie	nom de la voie	Varchar	NC	Elémentaire		
Total_piece	nombre de pièces	Integer	NC	Elémentaire		
Surface_carrez	surface selon loi Carrez	Float	NC	Elémentaire		
Surface_local	surface du local	Integer	NC	Elémentaire		
Type_local	type de local	Varchar	NC	Elémentaire		

Table Vente :

CODE	SIGNIFICATION	TYPE	LONGUEUR	NATURE	REGLE DE GESTION	REGLE DE CALCUL
id_vente	identifiant unique de la vente (PK)	Integer	NC	Incrémentation	Not null	1 à N
id_bien	identifiant unique du bien (FK)	Integer	NC	Incrémentation	Not null	1 à N
Date	date de la mutation	Date	10	Elémentaire	Not null	jj/mm/aaaa
Valeur	valeur du bien	Integer	NC	Elémentaire		

Le schéma relationnel normalisé (SQL Power Architect)

- 4 tables principales
- Relation assurée par des clés primaires et étrangères. Une vente est liée à un bien ; Un bien est rattaché à une commune ; Une commune appartient à une région
- Normalisation 3NF : Suppression des redondances, données organisées par entité, chaque information stockée une seule fois



La base de données avec les tables créées et les données chargées

Création des tables csv :

- 3 tables créées à partir des tables fournies (Bien, Vente, Commune)
- 1 table créée à partir d'un référentiel provenant de data.gouv (Region)
- Principales fonctions Excel utilisées : CHOISIRCOLS, CONCAT, RECHERCHEX

Modélisation :

- SQL Power Architect
- Création du schéma relationnel
- Définition des PK et FK

Déploiement du modèle :

- pgAdmin 4
- Import des csv
- Vérification et requêtage

Les requêtes ou screenshot qui permettent de démontrer le bon chargement des données

Chargement des tables :

Query | Query History

```
1 SELECT table_name
2 FROM information_schema.tables
3 WHERE table_schema = 'public';
```

Data Output | Messages | Notifications

	table_name name
1	region
2	commune
3	bien
4	vente

Nb de lignes par table :

Query | Query History

```
1 SELECT COUNT(*) FROM bien;
```

Data Output | Messages | Notifications

	count bigint
1	34169

Chargement des colonnes :

Query | Query History

```
1 SELECT column_name, data_type
2 FROM information_schema.columns
3 WHERE table_name = 'bien';
```

Data Output | Messages | Notifications

	column_name name	data_type character varying
1	no_voie	integer
2	total_piece	integer
3	surface_carrez	real
4	surface_local	integer
5	id_bien	integer
6	type_local	character varying
7	id_codedep_codecommune	character varying
8	btq	character varying
9	type_voie	character varying
10	voie	character varying

Requêtes SQL et résultats

Requête 1

Nombre total d’appartements vendus au 1er semestre 2020

	nb_appartements bigint
1	31378

```
SELECT COUNT(*) AS nb_appartements
FROM Vente v
JOIN Bien b ON v.id_bien = b.id_bien
WHERE b.type_local = 'Appartement'
AND v.date BETWEEN
TO_DATE('01/01/2020','DD/MM/YYYY')
AND TO_DATE('30/06/2020','DD/MM/YYYY');
```

- 31 378 appartements ont été vendus au 1er semestre 2020
- 2 tables concernées : bien et vente
- Jointure avec ‘bien[id_bien (PK)]’ et ‘vente[id_bien (FK)]’
- Fonctions principales : COUNT et WHERE

Requête 2

Le nombre de ventes d'appartements par région pour le 1er semestre 2020

nom_region	nb_ventes
Ile-de-France	13995
Provence-Alpes-Côte d'Azur	3649
Auvergne-Rhône-Alpes	3253
Nouvelle-Aquitaine	1932
Occitanie	1640
Pays de la Loire	1357
Hauts-de-France	1254
Grand Est	984
Bretagne	983
Normandie	862
Centre-Val de Loire	696
Bourgogne-Franche-Comté	376
Corse	223
Martinique	94
La Réunion	44
Guyane	34
Guadeloupe	2

```
SELECT r.nom_region,
COUNT(*) AS nb_ventes
FROM Vente v
JOIN Bien b ON v.id_bien = b.id_bien
JOIN Commune c ON b.id_codedep_codecommune =
c.id_codedep_codecommune
JOIN Region r ON c.code_region = r.code_region
WHERE b.type_local = 'Appartement'
AND v.date BETWEEN
TO_DATE('01/01/2020','DD/MM/YYYY')
AND TO_DATE('30/06/2020','DD/MM/YYYY')
GROUP BY r.nom_region
ORDER BY nb_ventes DESC;
```

- Nb de ventes le plus élevé en Ile-de-France (13 995 appartements).

  Le moins élevé en Guadeloupe avec 2 appartements.
- 4 tables concernées
- Fonctions principales : COUNT, WHERE et GROUPBY

Requête 3

Proportion des ventes d'appartements par le nombre de pièces

total_piece	nb_ventes	proportion_pct
0	30	0.10
1	6739	21.48
2	9783	31.18
3	8966	28.57
4	4460	14.21
5	1114	3.55
6	204	0.65
7	54	0.17
8	17	0.05
9	8	0.03
10	2	0.01
11	1	0.00

```
SELECT
b.total_piece,
COUNT(*) AS nb_ventes,
ROUND(100.0 * COUNT(*) / SUM(COUNT(*))
OVER(),2) AS proportion_pct
FROM Vente v
JOIN Bien b ON v.id_bien = b.id_bien
WHERE b.type_local = 'Appartement'
GROUP BY b.total_piece
ORDER BY b.total_piece;
```

- Les appartements 1,2 et 3 pièces sont les plus vendus et représentent entre 20 à 30% du total des ventes d'appartements.
- 2 tables concernées : bien et vente
- Fonctions principales : COUNT, SUM et GROUPBY

Requête 4

Liste des 10 départements où le prix du mètre carré est le plus élevé

code_departement	prix_m2
75	12052.82
92	7219.39
94	5343.28
6	4700.26
74	4667.13
93	4344.78
78	4225.25
69	4059.28
2A	4026.97
33	3764.14

```
SELECT
c.code_departement,
ROUND(AVG(v.valeur / b.surface_carrez)::numeric,2) AS
prix_m2
FROM Vente v
JOIN Bien b ON v.id_bien = b.id_bien
JOIN Commune c ON b.id_codedep_codecommune =
c.id_codedep_codecommune
WHERE b.surface_carrez > 0
AND v.valeur IS NOT NULL
GROUP BY c.code_departement
ORDER BY prix_m2 DESC
LIMIT 10;
```

- L'Ile de France affiche le prix au m² le plus élevé avec un prix moyen de 12 053 euros.
- 3 tables concernées : bien, vente et commune
- Fonctions principales : AVG, GROUPBY, ORDER BY et LIMIT

Requête 5

Prix moyen du mètre carré d'une maison en Île-de-France

	prix_m2_moyen numeric
1	3997.26

- Prix moyen d'une maison au m² en Ile de France : 3997 euros.
- 4 tables concernées
- Fonctions principales : AVG et WHERE

```
SELECT
ROUND(AVG(v.valeur / b.surface_local)::numeric,2) AS
prix_m2_moyen
FROM Vente v
JOIN Bien b ON v.id_bien = b.id_bien
JOIN Commune c ON b.id_codedep_codecommune =
c.id_codedep_codecommune
JOIN Region r ON c.code_region = r.code_region
WHERE b.type_local = 'Maison'
AND r.nom_region = 'Ile-de-France'
AND b.surface_local > 0
AND v.valeur IS NOT NULL;
```

Requête 6

Liste des 10 appartements les plus chers avec la région et le nombre de mètres carrés

valeur	surface_carrez	prix_m2	nom_region
9000000	9.1	989010.95	Ile-de-France
8600000	64	134375.00	Ile-de-France
8577713	20.55	417406.97	Ile-de-France
7620000	42.77	178162.26	Ile-de-France
7600000	253.3	30003.95	Ile-de-France
7535000	139.9	53859.90	Ile-de-France
7420000	360.95	20556.86	Ile-de-France
7200000	595	12100.84	Ile-de-France
7050000	122.56	57522.85	Ile-de-France
6600000	79.38	83144.37	Ile-de-France

```
SELECT
v.valeur,
b.surface_carrez,
ROUND((v.valeur / b.surface_carrez)::numeric,2) AS prix_m2,
r.nom_region
FROM Vente v
JOIN Bien b ON v.id_bien = b.id_bien
JOIN Commune c ON b.id_codedep_codecommune =
c.id_codedep_codecommune
JOIN Region r ON c.code_region = r.code_region
WHERE b.type_local = 'Appartement'
AND v.valeur IS NOT NULL
AND b.surface_carrez > 0
ORDER BY v.valeur DESC
LIMIT 10;
```

- Les 10 appartements les plus chers se situent en Ile-de-France.
- 4 tables concernées
- Fonctions principales : WHERE, ORDER BY, LIMIT

Requête 7

Taux d'évolution du nombre de ventes entre le premier et le second trimestre de 2020

	evolution_pct numeric
1	3.68

```
WITH ventes_trimestre AS (
SELECT
CASE
WHEN date BETWEEN TO_DATE('01/01/2020','DD/MM/YYYY')
    AND TO_DATE('31/03/2020','DD/MM/YYYY') THEN 'T1'
WHEN date BETWEEN TO_DATE('01/04/2020','DD/MM/YYYY')
    AND TO_DATE('30/06/2020','DD/MM/YYYY') THEN 'T2'
END AS trimestre
FROM Vente
WHERE date BETWEEN
TO_DATE('01/01/2020','DD/MM/YYYY')
AND TO_DATE('30/06/2020','DD/MM/YYYY')
)
SELECT
ROUND(
((SUM(CASE WHEN trimestre='T2' THEN 1 END) -
SUM(CASE WHEN trimestre='T1' THEN 1 END)) *100.0
/
SUM(CASE WHEN trimestre='T1' THEN 1 END))
,2) AS evolution_pct
FROM ventes_trimestre;
```

- Les ventes ont augmenté de 3,7% entre T1 et T2 2020.
- 1 table concernée (Vente)
- Fonctions principales table temporaire : WITH, CASE, WHEN, WHERE, END
- Fonctions principales de calcul : SUM, CASE…END

Requête 8

Le classement des régions par rapport au prix au mètre carré des appartement de plus de 4 pièces

nom_region	prix_m2
Ile-de-France	8770.44
La RÃ©union	3641.81
Provence-Alpes-CÃ´te d'Azur	3587.65
Corse	3104.88
Auvergne-RhÃ´ne-Alpes	2891.38
Nouvelle-Aquitaine	2465.48
Bretagne	2412.05
Pays de la Loire	2315.76
Hauts-de-France	2189.93
Occitanie	2097.23
Normandie	2015.77
Grand Est	1540.89
Centre-Val de Loire	1453.11
Bourgogne-Franche-ComtÃ©	1251.19
Martinique	573.48

```
SELECT
r.nom_region,
ROUND(AVG(v.valeur /
b.surface_carrez)::numeric,2) AS prix_m2
FROM Vente v
JOIN Bien b ON v.id_bien = b.id_bien
JOIN Commune c ON b.id_codedep_codecommune
= c.id_codedep_codecommune
JOIN Region r ON c.code_region = r.code_region
WHERE b.type_local = 'Appartement'
AND b.total_piece > 4
AND b.surface_carrez > 0
AND v.valeur IS NOT NULL
GROUP BY r.nom_region
ORDER BY prix_m2 DESC;
```

- Dans le segment appartement de plus de 4 pièces, le prix au m² est le plus élevé en Ile-de-France.
- 4 tables concernées
- Fonctions principales : AVG, WHERE, GROUP BY, ORDER BY

Requête 9

Liste des communes ayant eu au moins 50 ventes au 1er trimestre

nom_commune	nb_ventes
Paris 17e Arrondissement	228
Paris 15e Arrondissement	215
Paris 18e Arrondissement	209
Nice	173
Paris 11e Arrondissement	169
Paris 16e Arrondissement	165
Bordeaux	157
Paris 14e Arrondissement	146
Paris 20e Arrondissement	127
Nantes	119
Paris 19e Arrondissement	116
Paris 12e Arrondissement	110
Paris 10e Arrondissement	109
Paris 9e Arrondissement	106
Grenoble	106
Boulogne-Billancourt	99
Paris 13e Arrondissement	94
Paris 7e Arrondissement	87
Paris 6e Arrondissement	86
Marseille 8e Arrondissement	81
AsniÃ¨res-sur-Seine	81
Courbevoie	80
Paris 5e Arrondissement	79
Paris 3e Arrondissement	79
Toulouse	78
Antibes	77
Marseille 4e Arrondissement	72
Marseille 1er Arrondissement	71
Vincennes	68
Rueil-Malmaison	68
Lille	67
Marseille 9e Arrondissement	66
Montreuil	65
Angers	64
NÃ®mes	63
La Ciotat	62
Paris 8e Arrondissement	62
SÃ¨te	62
Rennes	61
Paris 2e Arrondissement	61
Paris 4e Arrondissement	60
Toulon	59
Levallois-Perret	59
Saint-Maur-des-FossÃ©s	56
Ajaccio	54
Versailles	54
Puteaux	53
Issy-les-Moulineaux	50

- 48 communes comptent 50 ventes ou plus au 1er trimestre 2020.
- Les communes de Paris sont majoritaires dans le top 10, dépassant 100 ventes, voire 200 selon les arrondissements.
- 3 tables concernées : vente, bien, commune
- Fonctions principales : COUNT, WHERE, GROUP BY, HAVING, ORDER BY

```
SELECT
c.nom_commune,
COUNT(*) AS nb_ventes
FROM Vente v
JOIN Bien b ON v.id_bien = b.id_bien
JOIN Commune c ON
b.id_codedep_codecommune =
c.id_codedep_codecommune
WHERE v.date BETWEEN
TO_DATE('01/01/2020','DD/MM/YYYY')
AND
TO_DATE('31/03/2020','DD/MM/YYYY')
GROUP BY c.nom_commune
HAVING COUNT(*) >= 50
ORDER BY nb_ventes DESC;
```

Requête 10

Différence en pourcentage du prix au m² entre un appartement de 2 pièces et un appartement de 3 pièces

	difference_pct numeric
1	14.16

- Le prix au m² d'un appartement 3 pièces est 14% plus élevé que le prix au m² d'un 2 pièces.
- 2 tables concernées : vente, bien
- Fonctions principales table temporaire : WITH, WHERE, GROUP BY
- Fonctions principales de calcul : MAX, MIN

```
WITH prix_m2 AS (
SELECT
b.total_piece,
AVG(v.valeur / b.surface_carrez) AS prix_m2
FROM Vente v
JOIN Bien b ON v.id_bien = b.id_bien
WHERE b.type_local = 'Appartement'
AND b.total_piece IN (2,3)
AND b.surface_carrez > 0
AND v.valeur IS NOT NULL
GROUP BY b.total_piece
)
SELECT
ROUND(((MAX(prix_m2) - MIN(prix_m2)) *100 /
MIN(prix_m2))::numeric,2) AS difference_pct
FROM prix_m2;
```

Requête 11

Les moyennes de valeurs foncières pour le top 3 des communes des départements 6, 13, 33, 59 et 69

code_departement	nom_commune	valeur_moyenne
6	Saint-Jean-Cap-Ferrat	968750.00
6	Eze	655000.00
6	Mouans-Sartoux	476898.13
13	Gignac-la-Nerthe	330000.00
13	Saint-Savournin	314425.00
13	Cassis	313416.88
33	LÃ¨ge-Cap-Ferret	549500.64
33	Vayres	335000.00
33	Arcachon	307435.93
59	Bersée	433202.00
59	Cysoing	408550.00
59	Halluin	322250.00
69	Ville-sur-Jarnioux	485300.00
69	Lyon 2e Arrondissement	455217.26
69	Lyon 6e Arrondissement	426968.25

- Les valeurs foncières les plus élevées se trouvent dans les départements 6 et 69.
- 3 tables concernées : vente, bien, commune
- Fonctions principales : WITH, WHERE, ROW_NUMBER, PARTITION BY, ORDER BY

```
WITH moyennes AS (
SELECT
c.code_departement,
c.nom_commune,
ROUND(AVG(v.valeur)::numeric,2) AS valeur_moyenne
FROM Vente v
JOIN Bien b ON v.id_bien = b.id_bien
JOIN Commune c ON b.id_codedep_codecommune =
c.id_codedep_codecommune
WHERE c.code_departement IN ('06','13','33','59','69')
AND v.valeur IS NOT NULL
GROUP BY c.code_departement, c.nom_commune
),
classement AS (
SELECT *,
ROW_NUMBER() OVER (PARTITION BY code_departement ORDER BY
valeur_moyenne DESC) AS rang
FROM moyennes
)
SELECT
code_departement,
nom_commune,
valeur_moyenne
FROM classement
WHERE rang <= 3
ORDER BY code_departement, rang;
```

Requête 12

Les 20 communes avec le plus de transactions pour 1000 habitants pour les communes qui dépassent les 10 000 habitants

nom_commune	transactions_pour_1000_habitants
Paris 2e Arrondissement	5.84
Paris 1er Arrondissement	4.92
Paris 3e Arrondissement	4.69
Arcachon	4.62
La Baule-Escoublac	4.58
Paris 4e Arrondissement	4.08
Roquebrune-Cap-Martin	3.99
Paris 8e Arrondissement	3.83
Sanary-sur-Mer	3.50
Paris 9e Arrondissement	3.43
La Londe-les-Maures	3.43
Paris 6e Arrondissement	3.38
Saint-Cyr-sur-Mer	3.24
Chantilly	3.13
Pornichet	3.06
Saint-MandÃ©	3.06
Paris 10e Arrondissement	3.04
Menton	2.94
Saint-Hilaire-de-Riez	2.87
Vincennes	2.81

```
SELECT
c.nom_commune,
ROUND((COUNT(v.id_vente) * 1000.0 /
c.population)::numeric,2) AS
transactions_pour_1000_habitants
FROM Vente v
JOIN Bien b ON v.id_bien = b.id_bien
JOIN Commune c ON b.id_codedep_codecommune =
c.id_codedep_codecommune
WHERE c.population > 10000
GROUP BY c.nom_commune, c.population
ORDER BY transactions_pour_1000_habitants DESC
LIMIT 20;
```

- Le plus grand nombre de transactions pour 1000 habitants sont sur Paris (5 transactions)
- 3 tables concernées : vente, bien, commune
- Fonctions principales utilisées : COUNT, WHERE, GROUP BY, LIMIT

Merci !